260602 Franka LLM Drawing Project Status Summary

Date: 2026-06-02 Purpose: Explain the current code structure, Isaac Sim execution flow, demo status, and future development direction to project teammates.

1. One-line project status

The project currently reaches the stage where a natural-language-like drawing command is converted into a `DrawingPlan` JSON, passed through a JADE-based robot execution layer, and executed in Isaac Sim with a Franka robot carrying a pen.

This is not the final complete system yet. The current demo uses a reproducible API-free `no-api` fallback instead of paid OpenAI API calls.

```
natural-language-like input
-> no-api fallback planner
-> DrawingPlan JSON
-> bridge validator
-> JADE trajectory/control
-> Isaac Sim Franka execution
-> logs and plots
```

2. Folder roles

```
Unit_Action_Langchain-main/
    Existing LLM planner layer.
    Its role is natural language -> DrawingPlan JSON.
    It does not handle IK, Jacobians, joint commands, or Isaac execution.

llm_isaac_bridge/
    Thin bridge added for this stage.
    It keeps the existing LLM planner and robot-control code separate.
    It handles natural-language-like input, plan generation, validation,
    and Isaac command generation.
    The current presentation demo uses the API-free no-api fallback.

franka_llm_drawing/
    Main robot-control core package currently used from the IR root.
    It converts DrawingPlan JSON into trajectories and handles frame transforms,
    pen-tip offsets, DLS IK, normal-force admittance, JADE evaluation, and Isaac execution.

configs/
    Frame, Isaac scene, JADE controller, and sampling configuration files.

usd/
    Isaac USD scene with Franka, table, paper, pen, and lights.

examples/
    Isaac runner scripts, offline sampling scripts, and sample plans.

outputs/
    Execution results, validation reports, plots, and CSV logs.

plan_and_information/
    Planning documents, control notes, frame audits, and demo notes.
```

3. Implemented core features

3.1 USD / Isaac scene

- The current scene is `usd/franka_drawing_scene_clean.usda`.
- Franka is referenced from a local project asset instead of an external URL.

- The scene contains table, paper, pen, and lights.
- The paper surface is aligned with `/World/BoardFrame`.
- The pen is attached to the Franka end-effector region.
- The controlled drawing target is `PenTipFrame`.
- Lighting was added so robot, table, paper, and pen motion are visible.

3.2 Commanded joints

The Isaac backend commands only the seven Franka arm joints.

```
panda_joint1
panda_joint2
panda_joint3
panda_joint4
panda_joint5
panda_joint6
panda_joint7
```

Finger joints may still exist in the USD articulation, but they are not command targets.

3.3 DrawingPlan handling

The planner or fallback creates a `DrawingPlan` JSON with symbolic unit actions.

```
move_to_start
align_pen_orientation
pen_down
draw_line
draw_arc
pen_up
```

These actions are not joint commands. They first go through the trajectory planner and become board-frame pen-tip pose samples.

3.4 Trajectory planning

The current trajectory layer reflects basic class-style trajectory concepts.

- Lines and arcs are sampled in time.
- Quintic time scaling is the default.
- Speed, acceleration, and jerk limits are applied.
- Drawing speed is intentionally conservative for tracking and contact stability.
- Pen-up release logic avoids sudden command jumps while contact is being released.

3.5 Controller / JADE

The current baseline is not a full torque controller. It uses a stable joint-position backend.

```
task-space DLS IK
+ pen-axis orientation task
+ tangent XY correction
+ normal-force admittance
+ JADE execution monitoring
```

JADE records and evaluates:

- planned vs actual XY path

- tracking error
- tip orientation error
- normal contact force
- force offset
- singular value
- condition number
- manipulability
- joint-limit margin

4. How the simulation runs

The current demo execution flow is:

```
1. The user enters a natural-language-like command.
   Example: "Draw a circle with radius 4 cm at the center."

2. llm_isaac_bridge runs the no-api fallback planner.
   It does not call the OpenAI API. It creates a deterministic DrawingPlan JSON.

3. The bridge validator checks the plan.
   It checks action order, board bounds, speed, pen state, and line/arc continuity.

4. The existing run_jade_isaac.py runner executes.
   It samples the DrawingPlan into a trajectory and transforms board frame to robot base
   ↪  frame.

5. Pen-tip offset compensation is applied.
   Desired pen-tip pose is converted into desired panda_link7 pose.

6. The Isaac backend reads Franka state and Jacobian.
   It reads q, qdot, end-effector pose, Jacobian, and contact force.

7. The controller creates targets for panda_joint1 through panda_joint7.
   It uses DLS IK and normal-force admittance.

8. Isaac Sim steps forward.
   The robot moves the pen over the paper on the table.

9. Results are saved under outputs.
   CSV logs, summary JSON, planned-vs-actual plots, force plots, and JADE metrics are
   ↪  generated.
```

5. API-free presentation demo inputs

The current presentation demo uses the following six fixed inputs.

case id	input	output
`circle_r4cm`	`중앙에 반지름 4cm짜리 원을 그려줘`	circle
`square_s6cm`	`중앙에 한 변 6cm짜리 사각형을 그려줘`	square
`triangle_s6cm`	`중앙에 한 변 6cm짜리 삼각형을 그려줘`	triangle
`letter_a_8cm`	`중앙에 알파벳 A를 8cm 크기로 그려줘`	letter A
`house_8cm`	`중앙에 8cm 집 모양을 그려줘`	house
`star_8cm`	`중앙에 8cm 별을 그려줘`	star

These are not true LLM reasoning results. They are deterministic rule-based fallback cases for a stable presentation demo.

A precise presentation statement is:

```
For the current demo, we use an API-free rule-based natural-language fallback
for reproducibility. The real LLM planner can be swapped in through the same
DrawingPlan schema.
```

6. How to run the demo

6.1 Generate plans and validation reports

```
cd /home/kimchangyeol/IsaacLab/IR
/home/kimchangyeol/IsaacLab/IR/llm_isaac_bridge/scripts/prepare_demo_showcase.sh
```

Generated files:

```
outputs/demo_showcase/plans/*.json
outputs/demo_showcase/reports/*_validation.json
outputs/demo_showcase/manifest.json
outputs/demo_showcase/isaac_commands.sh
```

6.2 Run one case in Isaac

```
cd /home/kimchangyeol/IsaacLab/IR

/home/kimchangyeol/IsaacLab/IR/llm_isaac_bridge/scripts/prepare_demo_showcase.sh    --case
↪  circle_r4cm    --execute-isaac
```

Available case ids:

```
square_s6cm
triangle_s6cm
letter_a_8cm
house_8cm
star_8cm
```

For recording a demo video, run one case at a time and record the Isaac Sim window manually.

6.3 Run all generated Isaac commands

```
bash /home/kimchangyeol/IsaacLab/IR/outputs/demo_showcase/isaac_commands.sh
```

For video recording, one-by-one execution is recommended over full batch execution.

7. What works now and what is still limited

Works now

- API-free natural-language-like input
- DrawingPlan JSON generation
- DrawingPlan validation
- Demo plans for circle, square, triangle, A, house, and star
- Isaac Sim execution of Franka pen-tip trajectories
- Contact force, tracking error, orientation error, and Jacobian metric logging
- Result plots and summary generation

Current limitations

- The no-api fallback is not a real LLM.
- It does not understand arbitrary natural language.
- The current fallback mostly supports centered drawings.
- Ink rendering on the paper is not implemented yet.
- Controller performance is still being improved.
- Real robot hardware execution is not implemented.
- Fully automatic OpenAI API planner integration requires separate API billing and clean environment separation.

8. Recommended future advanced features

8.1 LLM / Planner

- Connect a real OpenAI API or local LLM planner.
- Support more shapes and composed drawings.
- Support location phrases such as top-left, bottom-right, or multiple objects.
- Add stronger schema validation for LLM output.
- Add plan editing commands such as ”make it smaller” or ”move it right.”

8.2 Trajectory / Planning

- Automatic speed reduction around sharp corners.
- Stroke-order optimization for multi-stroke drawings.
- Combined paper-boundary and robot-reachability pre-check.
- Velocity and acceleration feedforward.
- SVG or handwriting path conversion.

8.3 Controller / JADE

- Reduce contact force spikes.
- Improve guarded pen-down.
- Stabilize normal-force admittance.
- Improve tangent tracking correction.
- Strengthen adaptive speed scaling near singularities and joint limits.
- Add optional torque hybrid force-position control as an advanced mode.
- Add more advanced null-space posture control.

8.4 Isaac / Visualization

- Automatic camera pose setup.
- Automatic screen recording or viewport capture.
- Ink trail rendering.
- Automatic report generation from plots.
- Demo case success/fail dashboard.

8.5 Collaboration / GitHub

- Share the IR project as an independent GitHub repository.
- Add no-api demo and Isaac run instructions to README.
- Prepare a dependency/setup guide for teammates.
- Prepare a presentation demo checklist.

9. Main message for teammates

1. The robot-control, Isaac scene, bridge, and JADE execution layers after the original LLM planner are now implemented separately.
2. The current system can show an API-free natural-language-like drawing demo in Isaac Sim.
3. Real LLM integration is structurally ready, but the presentation demo uses no-api fallback for cost and reproducibility.
4. The controller is not final; contact force and trajectory tracking will continue to improve.
5. The next team discussion should decide which advanced features matter most: real LLM, more shapes, stronger force control, automatic recording, or automated reports.